

Linux Embebido – Técnicas Digitales III (TD3)

Módulo 2

Kernel y Device Tree: fundamentos

Autor: Ing. Marín Ariel

Auxiliares becarios

Fabrizio Carlassara (BIDOC)

Pablo Gómez (BIDOC)

Alexis Hernández de la Cruz (BIS)

Resumen

Este módulo tiene dos objetivos complementarios. El primero es entender cómo se extiende el kernel de Linux: qué son los módulos cargables, cómo se compilan con Kbuild y cómo se insertan en el sistema en tiempo de ejecución, sin recompilar ni reiniciar el kernel. El segundo es entender el Device Tree: el mecanismo que usa Linux para describir el hardware que no puede descubrirse automáticamente, que es exactamente el caso de la Raspberry Pi y de la gran mayoría de los sistemas embebidos con procesadores ARM. Al terminar el módulo, el alumno habrá compilado y cargado su primer módulo de kernel directamente en la Pi, habrá visto el pipeline completo de Kconfig y Kbuild en acción aplicado a un módulo propio, e inspeccionado el Device Tree real de la placa.

Índice

1. Objetivos del módulo	2
2. El kernel de Linux: monolítico y extensible	2
2.1. Arquitectura: monolítico no significa rígido	2
2.2. ¿Qué puede hacer un módulo?	3
3. Kconfig y Kbuild	3
3.1. Kconfig: el origen de los símbolos CONFIG_*	3
3.2. Kbuild: el sistema de compilación	4
4. Práctica 1: el módulo <i>Hello World</i>	6
4.1. Instalación de las cabeceras del kernel	6
4.2. Estructura del módulo	6
4.3. Makefile	7
4.4. Compilación y carga	7

5. Práctica 2: Kconfig aplicado a un módulo <i>out-of-tree</i>	8
5.1. El archivo <code>Kconfig</code>	8
5.2. El archivo <code>.config</code>	8
5.3. Modificar el módulo y el <code>Makefile</code>	9
5.4. Verificación del flujo completo	10
5.5. (Opcional) Exploración con <code>menuconfig</code> via <code>kconfiglib</code>	10
6. Device Tree: describir el hardware que no se descubre solo	12
6.1. El problema: hardware no descubrible	12
6.2. Qué es el Device Tree	13
6.3. Sintaxis DTS	13
6.4. Del DTS al DTB: el compilador <code>dtc</code>	14
7. Práctica 3: inspeccionar el DTB de la Raspberry Pi	15
7.1. Instalación de <code>dtc</code>	15
7.2. Descompilar el DTB activo	15
7.3. Exploración guiada	15
8. Práctica 4: leer propiedades del Device Tree desde un módulo	15
8.1. La API <code>of_*</code>	16
8.2. El módulo	16
8.3. Compilación y prueba	17
9. Cierre conceptual	18

1. Objetivos del módulo

Al finalizar este módulo, el alumno debería poder:

- Explicar qué es un módulo cargable del kernel y en qué se diferencia del código compilado directamente en la imagen del kernel (*built-in*).
- Entender qué rol cumplen Kconfig y Kbuild, y cómo los símbolos `CONFIG_*` afectan la compilación tanto del kernel como de módulos propios.
- Escribir, compilar y cargar un módulo básico en la Raspberry Pi usando las herramientas nativas de la placa.
- Aplicar un archivo Kconfig a un módulo *out-of-tree* para condicionar código en tiempo de compilación.
- Explicar para qué existe el Device Tree y cuál es el problema concreto que resuelve en plataformas ARM embebidas.
- Leer y entender la sintaxis básica de un archivo DTS: nodos, propiedades, `compatible`, `reg`, `phandles` y `status`.
- Inspeccionar el Device Tree activo de la Raspberry Pi con `dtc`.

2. El kernel de Linux: monolítico y extensible

2.1. Arquitectura: monolítico no significa rígido

En el Módulo 1 se estableció que el kernel de Linux corre en el espacio de kernel, con acceso privilegiado al hardware, y que los drivers son parte de ese espacio. Una pregunta natural es en

qué sentido es “monolítico”: ¿implica que todo el código del kernel tiene que estar compilado en una sola imagen fija?

La respuesta es no. Linux es monolítico en el sentido arquitectónico: todo el código del kernel corre en el mismo espacio de direcciones, con el mismo nivel de privilegio, sin un mecanismo de paso de mensajes entre subsistemas como en un microkernel. Pero eso no implica que ese código tenga que estar fijo en tiempo de compilación. Linux permite cargar y descargar código del kernel *en tiempo de ejecución*, sin reiniciar el sistema, a través de los llamados **módulos cargables** (*loadable kernel modules*, o simplemente módulos) [1, cap. 4].

La Figura 1 ilustra la diferencia entre un driver integrado directamente en la imagen del kernel (*built-in*) y uno implementado como módulo cargable.

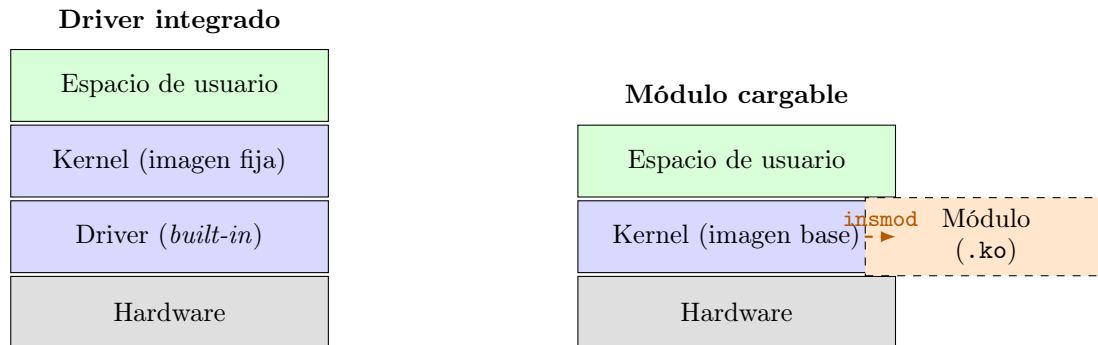


Figura 1: Comparación entre un driver compilado directamente en el kernel (*built-in*) y uno implementado como módulo cargable (*.ko*). En ambos casos el código corre en espacio de kernel con acceso pleno al hardware; la diferencia es cuándo y cómo se vincula al kernel en ejecución.

Un módulo de kernel es un archivo con extensión *.ko* (*kernel object*). Una vez compilado, puede insertarse en el kernel con `insmod` y removerse con `rmmmod`, sin reiniciar el sistema. Esto es especialmente valioso durante el desarrollo: compilar y reemplazar un módulo lleva segundos; recompilar todo el kernel y reiniciar la placa lleva minutos.

2.2. ¿Qué puede hacer un módulo?

Un módulo corre con los mismos privilegios que el resto del kernel. Puede:

- Acceder directamente a registros de hardware.
- Registrar un driver para un tipo de dispositivo.
- Crear archivos en `/dev` que las aplicaciones de espacio de usuario puedan abrir, leer y escribir (dispositivos de carácter; tema de los Módulos 4 y 5).
- Usar y exportar símbolos del kernel (funciones, variables) hacia otros módulos.

Y exactamente por eso, un bug en un módulo puede bloquear o corromper todo el sistema: no hay aislamiento de memoria entre el módulo y el resto del kernel. Es el mismo escenario conocido de un RTOS, donde un error en la capa de drivers puede corromper el scheduler o los datos de otras tareas.

3. Kconfig y Kbuild

3.1. Kconfig: el origen de los símbolos `CONFIG_*`

Al leer código del kernel o de módulos es común encontrar fragmentos como:

```
#ifndef CONFIG_GPIO_SYSFS
/* código que solo se compila si CONFIG_GPIO_SYSFS está activado */
#endif
```

Estos símbolos `CONFIG_*` no son constantes definidas a mano en el código: se generan a partir de un sistema de configuración llamado **Kconfig**. Su funcionamiento es el siguiente:

1. Cada subsistema del kernel tiene uno o más archivos **Kconfig** que declaran las opciones disponibles con su tipo (`bool`, `tristate`, `int`, `string`), sus dependencias entre sí, y un texto de ayuda.
2. Una herramienta de configuración (como `make menuconfig`) presenta esas opciones al desarrollador y genera un archivo `.config` con el resultado.
3. El sistema de compilación lee ese `.config` y genera un encabezado (`include/generated/autoconf.h`) que define cada opción activa como una macro de preprocesador.
4. En el código C, los `#ifndef CONFIG_*` controlan qué fragmentos se compilan según las selecciones hechas en el paso anterior.

La Figura 2 muestra ese flujo completo.

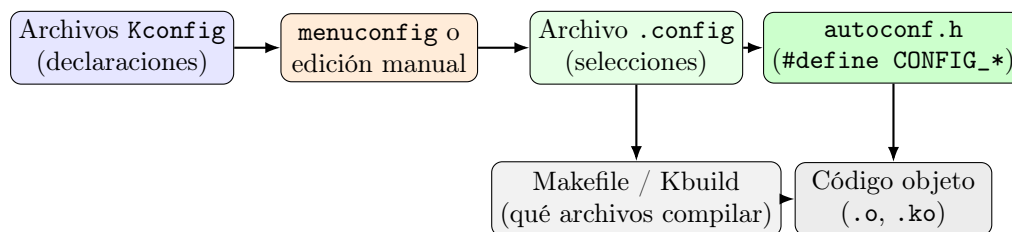


Figura 2: Pipeline de Kconfig: desde la declaración de opciones hasta el código objeto generado. El `.config` tiene dos efectos: define qué macros `CONFIG_*` se pasan al compilador (a través de `autoconf.h`) y, a través de Kbuild, determina qué archivos fuente se incluyen en la compilación.

Para este curso, lo más relevante no es cómo configurar el kernel completo (que tiene miles de opciones), sino entender *qué es un símbolo* `CONFIG_*` y de dónde viene. Los vamos a ver en el código de drivers del kernel a partir del Módulo 4, y los vamos a usar en nuestra propia práctica en la Sección 5.

3.2. Kbuild: el sistema de compilación

Kbuild es el sistema de compilación del kernel de Linux. A diferencia de un **Makefile** convencional, define variables y reglas especiales diseñadas para el contexto del kernel. Las más relevantes para el desarrollo de módulos son:

- `obj-y`: lista de archivos objeto que se compilan e integran directamente en la imagen del kernel (*built-in*).
- `obj-m`: lista de archivos objeto que se compilan como módulos cargables (`.ko`).
- `ccflags-y`: flags de compilación extra que se aplican a todos los archivos C del directorio.

El patrón `obj-$(CONFIG_FOO)` combina Kbuild con Kconfig: cuando `CONFIG_FOO=y` la variable expande a `obj-y` (integrado en el kernel); cuando `CONFIG_FOO=m`, a `obj-m` (módulo); cuando no está definida o vale `n`, a `obj-n` (no se compila). Este mecanismo es el núcleo del sistema de

configuración del kernel: el mismo código fuente produce resultados distintos según las selecciones en el `.config`.

Para un módulo desarrollado *fuera* del árbol de fuentes del kernel (*out-of-tree*), el Makefile del módulo delega todo el trabajo a la infraestructura de Kbuild del kernel ya instalado en la Pi:

```
obj-m += hello.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Cada línea del Makefile tiene una función específica que vale la pena entender en detalle:

- `obj-m += hello.o`: declara que `hello.o` (compilado desde `hello.c`) debe ensamblarse como módulo cargable. Si en lugar de `m` se usara `y`, el objeto se integraría directamente en la imagen del kernel. Esta variable la lee Kbuild, no `make` directamente; más abajo se explica cuándo.
- `KDIR := /lib/modules/$(shell uname -r)/build`: define la ruta al árbol de compilación del kernel instalado. La expansión `$(shell uname -r)` ejecuta `uname -r` en el shell y sustituye su salida por la versión del kernel en ejecución (por ejemplo, `6.18.34+rpt-rpi-v8`). El directorio resultante (`/lib/modules/<versión>/build`) es un enlace simbólico a las cabeceras del kernel instaladas, y es donde Kbuild busca sus reglas, definiciones de tipos y la configuración del kernel que se usó al compilarlo.
- `$(MAKE) -C $(KDIR) M=$(PWD) modules`: la regla central. Sus partes:
 - `$(MAKE)`: referencia al mismo binario `make` que está corriendo, incluyendo los flags con que fue invocado. Es la forma correcta de llamar a `make` recursivamente; llamar a `make` directamente podría usar una versión diferente o perder flags.
 - `-C $(KDIR)`: hace que `make` cambie su directorio de trabajo al árbol del kernel *antes* de leer ningún Makefile. Desde allí lee el Makefile principal de Kbuild, que es el que sabe cómo compilar módulos.
 - `M=$(PWD)`: le dice a Kbuild dónde está el código del módulo. Kbuild entra a ese directorio, lee *nuestro* Makefile por segunda vez (ahora desde el contexto de Kbuild, no desde el del usuario), y recoge la variable `obj-m` que declaramos al comienzo.
 - `modules`: el target de Kbuild para compilar módulos externos. Kbuild toma los `.c` listados en `obj-m`, los compila contra las cabeceras del kernel y genera el `.ko` final.

Nota

Nuestro Makefile es leído **dos veces**: la primera vez por el propio `make` cuando el usuario ejecuta `make all`, que lee la regla `all` e invoca `make -C $(KDIR)`; la segunda vez por Kbuild desde el contexto del kernel, para recoger `obj-m`. Esta doble invocación es un detalle de implementación de Kbuild: el Makefile actúa como punto de entrada para el usuario *y* como descriptor de módulo para Kbuild al mismo tiempo.

Nota

Los Makefiles en Linux requieren **tabulaciones** (no espacios) para indentar las recetas de cada regla. Si el editor convierte las tabulaciones en espacios al guardar, la compilación falla con el error `missing separator`. Verificar la configuración del editor antes de guardar el Makefile.

4. Práctica 1: el módulo *Hello World*

4.1. Instalación de las cabeceras del kernel

Para compilar un módulo en la Pi, se necesitan las cabeceras del kernel instalado. Sin ellas, Kbuild no puede encontrar las definiciones de tipos y funciones que usa el código del módulo. En algunas instalaciones de Raspberry Pi OS ya vienen incluidas; verificar antes de instalar:

```
# Verificar si ya están instaladas
ls /lib/modules/$(uname -r)/build/

# Si el directorio no existe o está vacío, instalar:
sudo apt update
sudo apt install raspberrypi-kernel-headers
```

Las cabeceras quedan bajo `/lib/modules/$(uname -r)/build/`, que es exactamente el directorio que el Makefile del módulo referencia con `KDIR`.

4.2. Estructura del módulo

Un módulo de kernel mínimo tiene tres componentes obligatorios: una función de inicialización, una función de limpieza, y macros de metadatos. Crear un directorio de trabajo en la Pi y dentro de él un archivo `hello.c`:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TD3 -- UTN FRA");
MODULE_DESCRIPTION("Modulo de ejemplo -- Modulo 2");

static int __init hello_init(void)
{
    printk(KERN_INFO "hello: modulo cargado\n");
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "hello: modulo descargado\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Detalles importantes sobre la anatomía del módulo:

- `MODULE_LICENSE("GPL")`: obligatorio. El kernel rechaza módulos sin licencia, y solo carga sin advertencias los que declaran una licencia compatible con GPL. La razón es que ciertos símbolos del kernel están marcados como `EXPORT_SYMBOL_GPL`: solo son accesibles para módulos con licencia GPL.

- `__init` y `__exit`: atributos que le indican al kernel que ese código puede liberarse de memoria luego de la inicialización (o del descargado). Son opcionales en un módulo mínimo pero se consideran buena práctica.
- `printk`: la función de impresión del kernel. No es `printf`: no existe la biblioteca estándar de C en el kernel. Los mensajes van al *kernel ring buffer* (un buffer circular en memoria) y se leen con `dmesg`.
- `KERN_INFO`: prefijo que define el nivel de prioridad del mensaje. Los niveles más usados son `KERN_ERR` (errores), `KERN_WARNING` (advertencias), `KERN_INFO` (información general) y `KERN_DEBUG` (depuración). El nivel determina si el mensaje aparece también en la consola del sistema o solo en `dmesg`.
- `module_init` y `module_exit`: registran las funciones de entrada y salida del módulo. El kernel las invoca al hacer `insmod` y `rmmod`; no se llaman directamente en el código.

Nota

La declaración de todas las macros y funciones usadas en este módulo puede consultarse directamente en el código fuente del kernel usando **Elixir Cross Referencer** [3]. Los encabezados relevantes son `include/linux/module.h` (macros de módulo, `module_init`, `MODULE_LICENSE`), `include/linux/kernel.h` (`printk`, niveles `KERN_*`) e `include/linux/init.h` (`__init`, `__exit`).

4.3. Makefile

En el mismo directorio que `hello.c`, crear el `Makefile`:

```
obj-m += hello.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

4.4. Compilación y carga

```
# Compilar el módulo
make

# Verificar el archivo generado
ls -lh hello.ko

# Cargar el módulo en el kernel
sudo insmod hello.ko

# Ver el mensaje de inicialización en el buffer del kernel
dmesg | tail -5

# Listar los módulos cargados y verificar que hello aparece
lsmod | grep hello

# Ver los metadatos del módulo
modinfo hello.ko
```

```
# Descargar el módulo
sudo rmmod hello

# Verificar el mensaje de limpieza
dmesg | tail -5
```

La Tabla 1 resume los comandos de gestión de módulos que vamos a usar a lo largo del curso.

Comando	Función
<code>insmod modulo.ko</code>	Carga el módulo especificado en el kernel.
<code>rmmod modulo</code>	Descarga el módulo (sin la extensión <code>.ko</code>).
<code>lsmod</code>	Lista todos los módulos actualmente cargados, su tamaño y los módulos que los referencian.
<code>modinfo modulo.ko</code>	Muestra los metadatos del módulo: licencia, autor, descripción, parámetros, dependencias.
<code>dmesg</code>	Muestra el contenido del buffer de mensajes del kernel.
<code>dmesg tail -10</code>	Muestra los últimos 10 mensajes (útil para ver el efecto de <code>insmod/rmmod</code>).
<code>dmesg -C</code>	Limpia el buffer de mensajes del kernel (requiere <code>sudo</code>).

Tabla 1: Comandos para gestionar módulos del kernel.

5. Práctica 2: Kconfig aplicado a un módulo *out-of-tree*

Con el módulo funcionando, extendemos el ejemplo para ver el pipeline completo de Kconfig: desde la declaración de una opción hasta un `#ifdef` en el código C, pasando por el `.config` y el Makefile.

5.1. El archivo Kconfig

Crear un archivo llamado `Kconfig` en el mismo directorio del módulo:

```
config HELLO_VERBOSE
    bool "Mensajes detallados al cargar el modulo"
    default n
    help
        Si se activa, el modulo imprime un mensaje adicional
        al cargarse con informacion sobre la configuracion
        de compilacion.
```

Este archivo declara una opción de tipo `bool` llamada `HELLO_VERBOSE`. El tipo `bool` solo admite dos estados: activo (`y`) o inactivo (`n`). Cuando esté activa, el sistema de Kconfig generará el símbolo `CONFIG_HELLO_VERBOSE` en el `.config` y, de ahí, una macro de preprocesador del mismo nombre en `autoconf.h`. Para un módulo *out-of-tree* vamos a reproducir ese pipeline manualmente, sin correr el kernel completo de Kconfig.

5.2. El archivo `.config`

En lugar de usar una herramienta gráfica, creamos el `.config` manualmente. El formato es el mismo que generaría `menuconfig`: un símbolo por línea, con `=y` si está activo, o como comentario si no lo está.

Con la opción desactivada:

```
# CONFIG_HELLO_VERBOSE is not set
```

Con la opción activada:


```
CONFIG_HELLO_VERBOSE=y
```

La convención de comentar la línea cuando la opción está desactivada (`# CONFIG_FOO is not set`) no es arbitraria: el propio `make` la interpreta como si la variable fuera `n`, lo que es útil cuando se usa la directiva `-include`.

5.3. Modificar el módulo y el Makefile

Actualizar `hello.c` para usar el símbolo `CONFIG_HELLO_VERBOSE`:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TD3 -- UTN FRA");
MODULE_DESCRIPTION("Modulo de ejemplo con Kconfig");

static int __init hello_init(void)
{
    printk(KERN_INFO "hello: modulo cargado\n");
#ifdef CONFIG_HELLO_VERBOSE
    printk(KERN_INFO "hello: compilado con verbose activo\n");
#endif
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "hello: modulo descargado\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Actualizar el Makefile para que lea el `.config` y transmita el símbolo al compilador:

```
-include .config

ccflags-$(CONFIG_HELLO_VERBOSE) += -DCONFIG_HELLO_VERBOSE

obj-m += hello.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Las dos líneas nuevas:

- `-include .config`: incluye el archivo `.config` si existe (el guion al comienzo hace que la directiva no falle si el archivo no existe). Al incluirlo, las variables `CONFIG_*` quedan definidas como variables de `make`.
- `ccflags-$(CONFIG_HELLO_VERBOSE) += -DCONFIG_HELLO_VERBOSE`: usa el patrón de Kbuild para flags condicionales. Cuando `CONFIG_HELLO_VERBOSE=y`, esta línea expande a `ccflags-y += -DCONFIG_HELLO_VERBOSE`, que le pasa la macro `-DCONFIG_HELLO_VERBOSE` al compilador para todos los archivos C del directorio. Cuando la variable no está definida o vale `n`, expande a `ccflags-n` (una variable que Kbuild ignora, equivalente a `no-op`).

5.4. Verificación del flujo completo

```
# Paso 1: opción desactivada
echo "# CONFIG_HELLO_VERBOSE is not set" > .config
make
sudo insmod hello.ko
dmesg | tail -5
# Resultado esperado: solo "hello: modulo cargado"
sudo rmmod hello

# Paso 2: opción activada
echo "CONFIG_HELLO_VERBOSE=y" > .config
make clean && make
sudo insmod hello.ko
dmesg | tail -5
# Resultado esperado: "modulo cargado" + "verbose activo"
sudo rmmod hello
```

Editar el `.config` cambia qué código se compila sin modificar el fuente del módulo. El flujo es idéntico al que usa el kernel completo con sus cientos de opciones: la escala es diferente, pero el mecanismo es el mismo.

5.5. (Opcional) Exploración con menuconfig via kconfiglib

La herramienta interactiva `menuconfig` que viene con el kernel (`mconf`, basada en `ncurses`) **no está incluida** en el paquete de cabeceras de Raspberry Pi OS: el directorio `scripts/kconfig/` del kernel instalado solo contiene `conf`, la variante no interactiva de línea de comandos. La alternativa es **kconfiglib**: una reimplementación completa en Python de todas las herramientas de `Kconfig`, incluyendo `menuconfig` con interfaz de texto interactiva. Lee los mismos archivos `Kconfig` y escribe el mismo formato `.config`, por lo que es completamente compatible con el pipeline que ya tenemos.

Instalación

`kconfiglib` se instala con `pip`, el gestor de paquetes de Python. En Raspberry Pi OS, `pip` puede no estar disponible por defecto y hay que instalarlo:

```
sudo apt install python3-pip
```

En versiones recientes de Raspberry Pi OS (basadas en Debian Bookworm), intentar instalar paquetes Python con `pip` directamente sobre el sistema da el error `externally-managed-environment`. Esto es una protección de Debian para evitar que paquetes instalados con `pip` rompan las dependencias del sistema operativo. La solución correcta es usar un **entorno virtual** (*virtual environment* o `venv`): un directorio aislado que contiene su propio intérprete de Python y sus propios paquetes, sin afectar al Python del sistema.

```
# Crear el entorno virtual (solo la primera vez)
# El nombre y la ubicación son libres; ~/venvs/kconfig es solo una convención
python3 -m venv ~/venvs/kconfig

# Activarlo (necesario en cada sesión nueva)
source ~/venvs/kconfig/bin/activate

# Instalar kconfiglib dentro del entorno activo
pip install kconfiglib
```

El directorio del entorno virtual puede estar en cualquier ubicación y tener cualquier nombre; `~/venvs/kconfig` es solo una convención legible. Lo importante es referenciar la misma ruta al activarlo. Cuando el entorno está activo, el prompt de la terminal muestra el nombre del entorno

entre paréntesis: (kconfig) lse@lse-pi4-00:~/hello_world\$. Para desactivarlo al terminar: deactivate.

Uso

Con el entorno activo, desde el directorio del módulo:

```
KCONFIG_CONFIG=.config menuconfig Kconfig
```

Cada parte del comando:

- `KCONFIG_CONFIG=.config`: variable de entorno que le indica a `kconfiglib` dónde leer y guardar el archivo de configuración. Si el `.config` ya existe, lo carga como estado inicial; si no existe, arranca con los valores `default` declarados en el `Kconfig`.
- `menuconfig`: el script instalado por `kconfiglib`, equivalente al `mconf` del kernel. Abre la interfaz de texto interactiva.
- `Kconfig`: el archivo de declaraciones que debe leer. `kconfiglib` lo busca en el directorio actual; si el archivo tuviera otro nombre o estuviera en otro directorio, se pasa la ruta aquí.

Navegación de la interfaz

La Figura 3 muestra la interfaz con la opción `HELLO_VERBOSE` activa (`[*]`). Los atajos de teclado disponibles aparecen en la barra inferior de la pantalla:

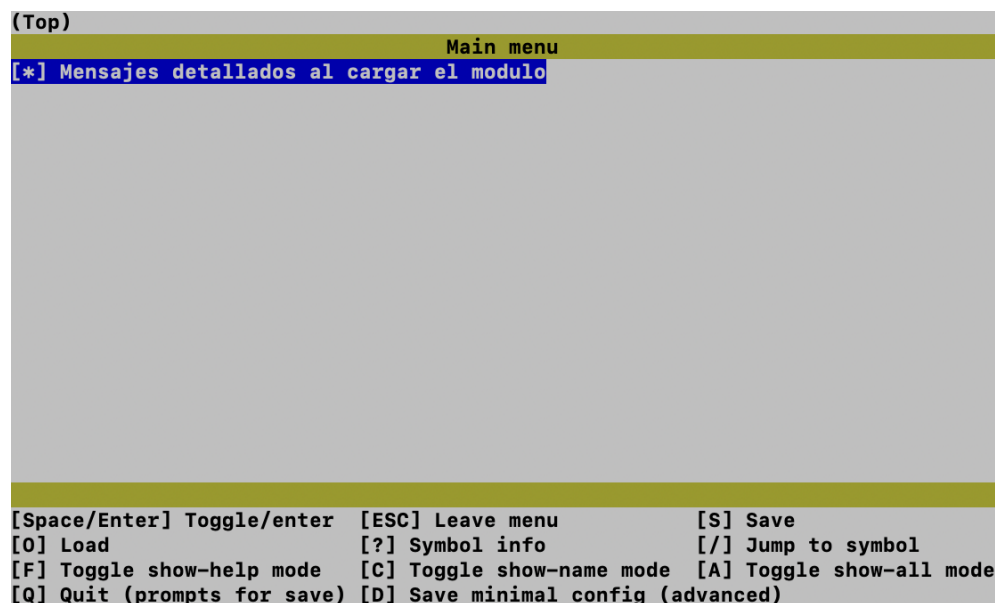


Figura 3: Interfaz `menuconfig` de `kconfiglib` con la opción `HELLO_VERBOSE` activada (`[*]`). La barra superior muestra la posición en el árbol de menús ((Top) >Main menu); la barra inferior lista los atajos de teclado disponibles.

Una opción `bool` activada se muestra como `[*]`; desactivada, como `[ ]`. Al salir con `Q` y confirmar, el `.config` queda actualizado con el mismo formato que la edición manual, y el `Makefile` lo usa en la siguiente compilación sin ningún cambio.

Tecla	Acción
Flechas ↑/↓	Moverse entre opciones.
Space / Enter	Activar o desactivar una opción bool (alterna entre [*] y []); entrar a un submenú.
ESC	Salir del submenú actual y volver al nivel anterior.
S	Guardar el <code>.config</code> sin salir.
Q	Salir (pregunta si guardar antes de cerrar).
?	Mostrar información detallada de la opción seleccionada: nombre del símbolo <code>CONFIG_*</code> , tipo, valor actual y texto de ayuda.
/	Buscar una opción por nombre de símbolo (útil cuando hay muchas opciones).
C	Alternar entre mostrar solo la descripción y mostrar también el nombre del símbolo <code>CONFIG_*</code> junto a cada opción.
A	Mostrar u ocultar opciones que no son visibles por defecto (por ejemplo, opciones cuya dependencia no se cumple).

Tabla 2: Atajos de teclado de la interfaz `menuconfig` de `kconfiglib`.

6. Device Tree: describir el hardware que no se descubre solo

6.1. El problema: hardware no descubrible

En una PC con bus PCI o con USB, el hardware se puede *enumerar*: al encender el equipo, el sistema operativo pregunta qué dispositivos hay en el bus y estos responden con su identificación (fabricante, producto, clase). A partir de esa información el kernel carga el driver correspondiente de forma automática.

En sistemas embebidos con procesadores ARM, la situación es distinta. La mayoría de los periféricos están conectados directamente al bus interno del SoC¹ mediante interfaces como UART, SPI, I2C, o GPIO mapeado en memoria. Estos buses no tienen mecanismo de enumeración: si se pregunta “¿qué hay conectado en la dirección `0x7e201000`?”, el bus no responde. Esa información la conoce el diseñador del hardware, no el sistema en tiempo de ejecución.

El problema del hardware no descubrible no es exclusivo de ARM. Existe en cualquier arquitectura donde los periféricos se conectan directamente al bus interno del SoC sin protocolo de enumeración: PowerPC, MIPS, RISC-V y otros enfrentan el mismo escenario. La diferencia es que PowerPC y SPARC ya tenían una solución antes de que Linux la necesitara: el estándar *Open Firmware* [2], desarrollado originalmente por Sun Microsystems para sus estaciones de trabajo SPARC a mediados de los años 80 y adoptado también por IBM y Apple en sus plataformas PowerPC durante los 90. Open Firmware define, entre otras cosas, un formato de árbol de nodos para describir el hardware de la plataforma, que el firmware pasa al sistema operativo al arrancar. En Linux, el soporte para este árbol en arquitecturas PowerPC y SPARC existía mucho antes de que el problema se volviera generalizado; ese formato es el antepasado directo del Device Tree actual [1, cap. 3].

El problema se volvió agudo con la explosión de plataformas ARM embebidas de los años 2000 y 2010. A diferencia de PowerPC, ARM no tenía ninguna convención de firmware que describiera el hardware, y cada fabricante de SoC resolvía el problema a su manera: código específico de cada placa en `arch/arm/mach-*/board.c`, con funciones de inicialización que duplicaban estructuras casi idénticas entre sí para cada variante de hardware. El árbol de fuentes del kernel acumuló cientos de archivos de este tipo hasta que, en enero de 2011, Linus Torvalds publicó en la lista de correo del kernel su diagnóstico del estado de `arch/arm/mach-*`: consideró la situación un “completo desastre” y señaló que el soporte ARM en el kernel estaba siendo manejado de una forma que no escalaba [4]. Ese fue el detonante definitivo para adoptar el formato de Device Tree

¹*System on a Chip*: integrado que contiene el procesador, la memoria y los periféricos en un solo chip.

—heredado y extendido desde Open Firmware— de forma sistemática en todo el kernel, no solo para ARM sino como mecanismo estándar para cualquier arquitectura sin enumeración de bus [1, cap. 3].

La solución adoptada fue mover esa descripción fuera del código C y a un formato de datos separado: el **Device Tree**.

6.2. Qué es el Device Tree

Un Device Tree es un árbol de nodos que describe la topología de hardware de una placa: qué periféricos existen, en qué dirección de memoria están mapeados sus registros, a qué interrupción están conectados, de qué reloj dependen, y qué driver debe manejarlos. Se escribe en un lenguaje llamado **DTS** (*Device Tree Source*) y se compila a un formato binario compacto llamado **DTB** (*Device Tree Blob*) que el bootloader carga en memoria y pasa al kernel al arrancar [1, cap. 3].

El kernel usa la información del DTB para dos cosas fundamentales:

1. Conocer qué hardware existe y cómo está configurado (direcciones de registros, líneas de interrupción, fuentes de reloj, pines, etc.).
2. Vincular cada nodo del árbol con su driver, a través de la propiedad `compatible`.

6.3. Sintaxis DTS

Un archivo DTS tiene estructura de árbol. El siguiente fragmento muestra la anatomía de un nodo típico, representativo de lo que se puede encontrar en el DTB real de la Raspberry Pi:

```
/dts-v1/;

/ {
    /* Nodo raiz. Todo el arbol cuelga de aqui. */

    model = "Raspberry Pi 4 Model B";
    compatible = "raspberrypi,4-model-b", "brcm,bcm2711";

    #address-cells = <1>;
    #size-cells = <1>;

    soc {
        /* Nodo que agrupa los perifericos del SoC */
        compatible = "simple-bus";
        #address-cells = <1>;
        #size-cells = <1>;

        uart0: serial@7e201000 {
            /* Nodo del UART0 del BCM2711 */
            compatible = "arm,pl011", "arm,primecell";
            reg = <0x7e201000 0x200>;
            /* base tamaño */
            interrupts = <2 25>;
            clocks = <&clk_uart0>;
            status = "okay";
        };
    };

    clk_uart0: clk-uart0 {
        compatible = "fixed-clock";
        #clock-cells = <0>;
        clock-frequency = <48000000>;
    };
};
```

Los elementos clave de la sintaxis:

- **Nodos:** `nombre@unidad-de-dirección { ... }`. La parte después del @ es convencional: para periféricos de memoria, es la dirección base de sus registros en hexadecimal.
- **Propiedad compatible:** la más importante. Es una lista de strings ordenada de más específico a más genérico. El kernel recorre la lista y carga el primer driver registrado para alguno de esos strings. Para el UART del ejemplo, el kernel buscaría primero un driver registrado para `"arm,pl011"` y, si no encuentra ninguno, uno para `"arm,primecell"`.
- **Propiedad reg:** describe el espacio de registros del periférico en memoria. El formato `<dirección tamaño>` lo determinan las propiedades `#address-cells` y `#size-cells` del nodo padre.
- **Etiquetas (labels):** `uart0:` o `clk_uart0:` son etiquetas que permiten referenciar un nodo desde otro lugar del árbol. La referencia `&clk_uart0` en la propiedad `clocks` del UART es un *phandle*: le indica al kernel que ese nodo UART debe obtener su reloj del nodo `clk_uart0`.
- **Propiedad status:** controla si el periférico está habilitado (`"okay"`) o deshabilitado (`"disabled"`). En el Device Tree base de la Pi, muchos UARTs secundarios están con `status = "disabled"`; el mecanismo de overlays (Módulo 6) los activa sobrescribiendo esta propiedad.
- **Propiedades #address-cells y #size-cells:** le dicen al kernel cuántas celdas de 32 bits componen la dirección y el tamaño en los nodos hijos. Son parte del “idioma” del Device Tree para describir el espacio de direcciones.

Nota

En este módulo la sintaxis DTS se presenta solo como referencia para poder leer lo que devuelve `dtc`. La escritura real de archivos DTS se aborda en el Módulo 6, cuando se crea un overlay. Para quienes quieran profundizar, la especificación completa está en <https://www.devicetree.org/specifications/> y el Cap. 3 del libro de referencia cubre la sintaxis con más detalle [1, cap. 3].

6.4. Del DTS al DTB: el compilador `dtc`

El compilador de Device Tree es `dtc` (*Device Tree Compiler*). Convierte el archivo de texto `.dts` al binario `.dtb`, y también en sentido inverso, que es exactamente lo que vamos a aprovechar en la práctica para leer el DTB de la Pi.

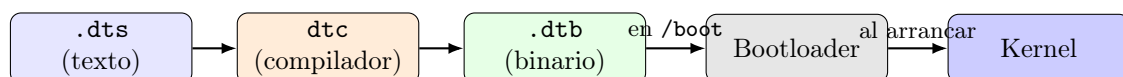


Figura 4: Pipeline del Device Tree: desde el archivo fuente hasta el kernel. El bootloader carga el `.dtb` en memoria y le pasa su dirección al kernel como parte de los parámetros de arranque. El kernel parsea el DTB y vincula cada nodo con su driver.

En la Raspberry Pi, los DTBs precompilados para cada modelo de placa están en `/boot/firmware/` (Raspberry Pi OS Bookworm en adelante) o en `/boot/` (Buster y Bullseye). El bootloader de la Pi selecciona automáticamente el DTB correcto según el modelo de placa detectado al encender el equipo:

```
ls /boot/firmware/*.dtb
```

7. Práctica 3: inspeccionar el DTB de la Raspberry Pi

7.1. Instalación de dtc

```
sudo apt install device-tree-compiler
```

7.2. Descompilar el DTB activo

La forma más legible de ver el Device Tree que el kernel está usando es descompilar el DTB binario de vuelta a texto DTS:

```
# Para Raspberry Pi 4
dtc -I dtb -O dts /boot/firmware/bcm2711-rpi-4-b.dtb 2>/dev/null | less

# Para Raspberry Pi 5
dtc -I dtb -O dts /boot/firmware/bcm2712-rpi-5-b.dtb 2>/dev/null | less
```

Las opciones de `dtc`:

- `-I dtb`: especifica que la entrada es un DTB binario.
- `-O dts`: especifica que la salida debe ser DTS (texto legible).
- `2>/dev/null`: descarta los mensajes de advertencia de `dtc`, que son normales con DTBs de producción que usan extensiones propietarias del fabricante.

7.3. Exploración guiada

Con el DTB descompilado abierto en `less` (usar `/` para buscar y `q` para salir), explorar los siguientes elementos:

```
# Buscar nodos de UART (notar la propiedad compatible y el status)
/uart

# Buscar la propiedad compatible del nodo raiz (modelo de placa)
/compatible

# Ver los overlays de Device Tree disponibles para la Pi
ls /boot/firmware/overlays/ | head -20
```

El directorio `/boot/firmware/overlays/` contiene los *Device Tree Overlays*: DTBs parciales que modifican o extienden el Device Tree base en tiempo de arranque. En el Módulo 6 vamos a crear y aplicar un overlay propio para habilitar un UART secundario dedicado a la comunicación con la placa ESP32-S3.

Nota

El Device Tree que el kernel está usando en tiempo real también es accesible a través del pseudo-sistema de archivos `/sys/firmware/devicetree/base/`: cada nodo del árbol aparece como un directorio y cada propiedad como un archivo. Por ejemplo, ejecutar `cat /sys/firmware/devicetree/base/model` imprime el modelo de la placa. Es una forma de navegar el Device Tree con las herramientas de archivos habituales, sin necesidad de `dtc`.

8. Práctica 4: leer propiedades del Device Tree desde un módulo

Ya se inspeccionó el Device Tree desde espacio de usuario con `dtc` y `/sys/firmware/devicetree/`. En esta práctica se hace lo mismo desde espacio de kernel, usando la API `of_*` (*Open Firmware*) que usa cualquier driver real cuando necesita saber cómo está configurado su periférico.

8.1. La API of_*

El kernel expone el Device Tree a los drivers a través de funciones declaradas en `<linux/of.h>`. Las más fundamentales son:

- `of_find_node_by_path(path)`: devuelve el nodo en la ruta indicada (por ejemplo, "/" para el nodo raíz).
- `of_find_compatible_node(from, type, compatible)`: busca el primer nodo cuya propiedad `compatible` contenga el string dado. Pasando `NULL` en los primeros dos argumentos busca desde el principio del árbol. Es exactamente lo que hace el driver core automáticamente cuando hace *matching* entre un nodo del DT y un driver.
- `of_property_read_string(node, propname, &out)`: lee una propiedad de tipo string. Devuelve 0 si tuvo éxito.
- `of_device_is_available(node)`: devuelve `true` si el nodo tiene `status = "okay"` (o si la propiedad `status` no está presente, lo que también significa habilitado).
- `of_node_put(node)`: libera la referencia al nodo. Toda función `of_find_*` incrementa un contador de referencias; `of_node_put` lo decrementa. Olvidarlo produce una pérdida de recursos en el kernel.

Nota

La API `of_*` completa está declarada en `include/linux/of.h`. Para consultarla en el código fuente del kernel usar Elixir Cross Referencer [3]: buscar el archivo `include/linux/of.h` en la versión más cercana a la instalada (`uname -r`), o buscar directamente el nombre de la función en el campo de búsqueda de identificadores para ver su declaración, implementación y todos los sitios donde se la usa en el árbol del kernel.

8.2. El módulo

Crear un directorio nuevo `dt_reader/` con los siguientes archivos:

`dt_reader.c`:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/of.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TD3 -- UTN FRA");
MODULE_DESCRIPTION("Lectura de propiedades del Device Tree desde kernel");

static int __init dt_reader_init(void)
{
    struct device_node *node;
    const char *str;

    /* Leer el modelo de placa desde el nodo raíz */
    node = of_find_node_by_path("/");
    if (node) {
        if (of_property_read_string(node, "model", &str) == 0)
            printk(KERN_INFO "dt_reader: modelo: %s\n", str);
        of_node_put(node);
    }

    /* Buscar el primer nodo compatible con "arm,pl011" (UART) */
    node = of_find_compatible_node(NULL, NULL, "arm,pl011");
```



```

if (node) {
    /* %pOF imprime la ruta completa del nodo en el arbol */
    printk(KERN_INFO "dt_reader: nodo: %pOF\n", node);
    printk(KERN_INFO "dt_reader: habilitado: %s\n",
            of_device_is_available(node) ? "si" : "no");
    of_node_put(node);
} else {
    printk(KERN_INFO "dt_reader: no se encontro nodo arm,pl011\n");
}

return 0;
}

static void __exit dt_reader_exit(void)
{
    printk(KERN_INFO "dt_reader: modulo descargado\n");
}

module_init(dt_reader_init);
module_exit(dt_reader_exit);

```

Makefile:

```

obj-m += dt_reader.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean

```

8.3. Compilación y prueba

```

make
sudo insmod dt_reader.ko
dmesg | tail -5
sudo rmmod dt_reader

```

La salida esperada en `dmesg` es algo como:

```

dt_reader: modelo: Raspberry Pi 4 Model B
dt_reader: nodo: /soc/serial@7e201000
dt_reader: habilitado: si

```

El nodo `/soc/serial@7e201000` es exactamente el que se vio al descompilar el DTB con `dtc` en la Práctica 3: el mismo árbol que se exploró desde espacio de usuario ahora se atraviesa desde dentro del kernel con las mismas funciones que usa cualquier driver real. La ruta y el estado `habilitado: si` coinciden con la propiedad `status = "okay"` del DTS.

Nota

El especificador de formato `%pOF` es una extensión del `printk` del kernel para imprimir la ruta completa de un nodo del Device Tree. No está disponible en `printf` de espacio de usuario; es exclusivo de código de kernel. La lista completa de especificadores propios del kernel está documentada en `Documentation/core-api/printk-formats.rst` dentro del árbol de fuentes, consultable en Elixir [3].

9. Cierre conceptual

Este módulo estableció dos mecanismos del kernel que van a aparecer en todos los módulos siguientes:

- Los **módulos cargables** permiten extender el kernel en tiempo de ejecución. En los Módulos 4 y 5 vamos a escribir un driver de dispositivo de carácter como módulo, que crea un nodo en `/dev` y permite que las aplicaciones de espacio de usuario controlen hardware real usando las llamadas al sistema habituales (`open`, `read`, `write`, `ioctl`).
- El **Device Tree** describe el hardware disponible y vincula cada periférico con su driver a través de `compatible`. En la Práctica 4 se usó la API `of_*` para leer propiedades del árbol desde kernel, que es exactamente lo que hace un driver real cuando se inicializa. En el Módulo 6 se va a escribir un overlay que habilita un UART secundario de la Pi; en el Módulo 7, ese UART será el canal de comunicación con la placa ESP32-S3 corriendo FreeRTOS.

Referencias

- [1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.^a edición. Packt Publishing.
 - Cap. 3, *All about Bootloaders* – Device Tree: origen del problema, sintaxis DTS, compilación con `dtc`, Device Tree Overlays.
 - Cap. 4, *Configuring and Building the Kernel* – Kconfig, Kbuild, módulos de kernel: `insmod/rmmod/dmesg`, compilación de módulos *out-of-tree*.
- [2] IEEE Std 1275-1994. *IEEE Standard for Boot (Initialization Configuration) Firmware: Core Requirements and Practices*. Institute of Electrical and Electronics Engineers, 1994. Estándar que define el formato de árbol de Open Firmware, antepasado directo del Device Tree de Linux.
- [3] Bootlin. *Elixir Cross Referencer – Linux source code*. <https://elixir.bootlin.com/linux/latest/source>. Herramienta de navegación del código fuente del kernel de Linux con referencias cruzadas: permite buscar archivos de cabecera, funciones, macros y tipos, y ver todos los sitios del árbol donde se los define o referencia. Seleccionar la versión más cercana a la del kernel instalado (`uname -r`) para obtener la definición exacta de cada símbolo.
- [4] Torvalds, L. (2011). Mensaje en la lista de correo *linux-arm-kernel* y *linux-kernel*, enero de 2011. Hilo: “[PATCH 1/4] ARM: remove mach/timex.h for plat-nomadik”. Archivado en lkml.org y marc.info. En ese intercambio, Torvalds señaló que el estado del soporte ARM en el árbol de fuentes del kernel era un “completo desastre” (“*a f*cking pain in the ass*” en el texto original) y cuestionó la proliferación de archivos de configuración de placa casi idénticos, lo que aceleró la adopción de Device Tree en la arquitectura ARM.